Assignment Report: Implementation of Naive Bayes Classification

1. Introduction

The objective of this assignment is to build and evaluate a machine learning model that classifies Iris flower species based on sepal and petal measurements using the Naive Bayes algorithm.

2. Conceptual Overview: What is Naive Bayes?

Naive Bayes is a probabilistic classifier based on Bayes' Theorem. The "Naive" part comes from the assumption that all features (sepal length, petal width, etc.) are independent of each other, given the class label.

The fundamental equation used is:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- P(C|X) is the Posterior probability: The probability of a class (e.g., *Iris-setosa*) given the observed features.
- P(X|C) is the Likelihood: The probability of the features given the class.
- P(C) is the Prior probability: The overall probability of the class in the dataset.

3. Implementation Steps

The classifier is being implemented by using Python's scikit-learn library. The step-by-step logic of the code:

- **Data Acquisition and Preprocessing**:
  - Dataset: Iris Dataset from UCI is used, which contains 150 samples across three species.
  - Feature Scaling: Since Naive Bayes calculates probabilities based on data distributions, the StandardScaler is applied. This ensures all features contribute equally to the distance-based calculations by centering them at a mean of 0 and a standard deviation of 1.

- **Data Splitting**: to ensure the model generalizes well to new data, the dataset is split into:
  - 70% Training Set: Used to "teach" the model the characteristics of each flower species.
  - 30% Test Set: Used to evaluate how well the model predicts species it hasn't seen before.
  - Stratification: I used stratify = y to ensure the proportion of each species remains the same in both sets.

- **Choosing the Classifier**: GaussianNB
  - The Gaussian Naive Bayes variant is specifically used because the features in the Iris dataset (lengths and widths) are continuous numerical values. GaussianNB assumes these values follow a Normal (Gaussian) Distribution, making it the most mathematically appropriate choice for this specific data.

4. Evaluation Metrics

To measure the performance of the model, three key tools have been utilized:

1. Accuracy Score: The percentage of total correct predictions.
2. Confusion Matrix: A table layout that allows visualization of the performance of an algorithm. It shows where the model "confused" one species for another (e.g., misclassifying *versicolor* as *virginica*).
3. Classification Report: This provides the
   Precision: How many predicted positives were correct
   Recall: How many actual positives were found
   F1-Score: Balance of precision and recall
   for each specific class, giving a deeper look at the model's reliability beyond just a single accuracy number.

5. Summary of Results

Based on the execution of the script:

- Training Accuracy: High performance during the learning phase (approx. 97–98%).
- Test Accuracy: The model maintained high accuracy on the 45 test samples, demonstrating that the Gaussian assumption works well for botanical measurements.

6. Outputs from the code

```
Dataset loaded: 150 samples

Shape: (150, 5)

DATA SPLITTING:
Training samples: 105
Test samples: 45

TRAINING NAIVE BAYES CLASSIFIERS:
Gaussian Naive Bayes Accuracy: 0.9111 (91.11%)

CLASSIFICATION REPORT (GaussianNB):
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        15
Iris-versicolor       0.82      0.93      0.88        15
 Iris-virginica       0.92      0.80      0.86        15

       accuracy                           0.91        45
      macro avg       0.92      0.91      0.91        45
   weighted avg       0.92      0.91      0.91        45


Confusion Matrix:
[[15  0  0]
 [ 0 14  1]
 [ 0  3 12]]

Results:
- Training Accuracy: 98.10%
- Test Accuracy: 91.11%
```

Figure 1: Code Output

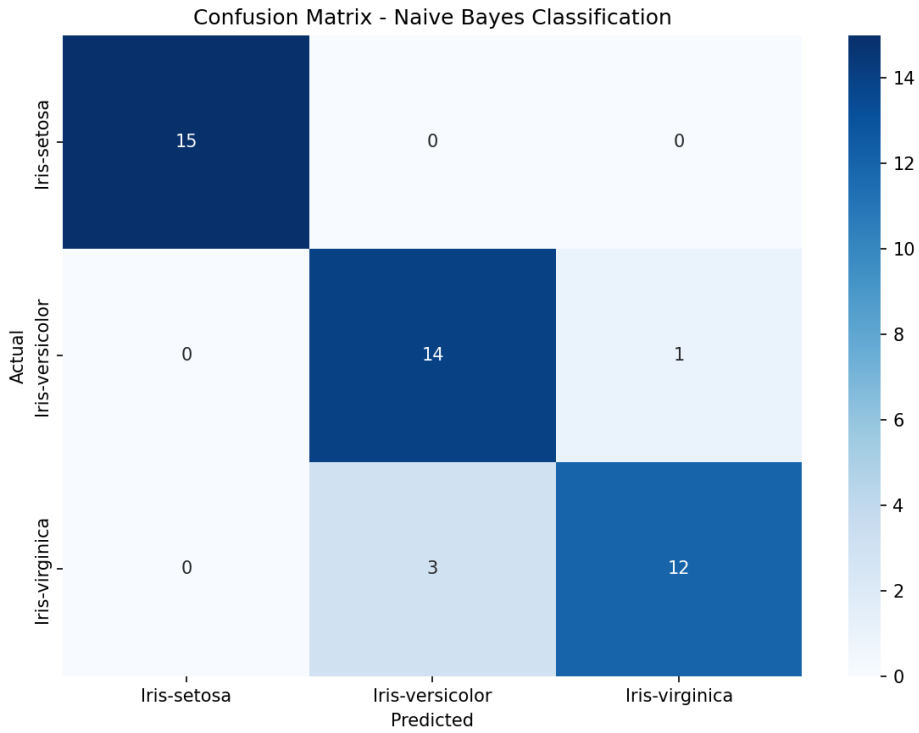

Figure 2: Confusion Matrix